PolicyPilot

Smart Government Scheme Assistant for Citizens

Product Requirements Document v1.0 — Hackathon Edition

Field	Details
Product name	PolicyPilot
Version	1.0 — Hackathon MVP
Date	2025
Authors	Team PolicyPilot
Problem domain	Civic tech / GovTech / AI for social impact
Target users	Indian citizens, CSC operators, NGO caseworkers
Primary market	India — Central + State government schemes
Tech stack	Python (FastAPI), React, RAG (ChromaDB/FAISS), Gemini API, SQLite

1. Executive Summary

India has over 450 central and state government welfare schemes — covering agriculture, housing, education, health, employment, and more. Yet fewer than 15% of eligible citizens successfully apply for schemes they qualify for. The gap is not desire; it is friction. Citizens don't know what schemes exist, can't understand eligibility rules written in bureaucratic language, don't know what documents to gather, and get lost in multi-portal application flows.

PolicyPilot is an AI-powered government scheme assistant that turns this broken process into a guided, end-to-end experience. A citizen describes their situation in plain language or uploads their documents. PolicyPilot analyzes real government scheme PDFs using Retrieval-Augmented Generation (RAG) to find every scheme they qualify for, explains exactly why with citations from the source documents, flags conflicts between schemes, and generates a personalized application checklist with direct portal links.

> **Core value proposition**
> Not just scheme discovery — end-to-end guidance from 'I don't know what I qualify for' to 'Here is your pre-filled application ready to submit.'

2. Problem Statement

2.1 The access gap

India's welfare delivery system suffers from a fundamental last-mile failure. Schemes are designed in ministry offices and announced in gazettes that no ordinary citizen reads. By the time awareness reaches a rural farmer or urban daily-wage worker, application windows have often closed, or the citizen lacks one document and doesn't know where to get it.

Pain point	Current reality	Impact
Discovery	Citizens must visit scheme portals one by one or rely on word of mouth	~85% of eligible citizens unaware of applicable schemes
Understanding	Eligibility criteria written in legal/bureaucratic language	Citizens can't self-assess whether they qualify
Documentation	No centralized list of required documents per scheme	Applications rejected for missing paperwork
Application	Each scheme has a different portal, login, and form format	High dropout at application stage
Tracking	No unified status tracking across schemes	Citizens don't know if application was received

2.2 Who is most affected

- Rural citizens with limited digital literacy — highest need, lowest access
- Women, especially widows and single mothers — large scheme coverage but low awareness

- Farmers — complex interlocking schemes (PMFBY, PM-KISAN, PMKSY) with interdependencies
- Urban informal workers — migrant schemes, self-employment support, health coverage
- Students from lower-income families — scholarship and fee-waiver schemes routinely missed

3. Goals & Success Metrics

3.1 Product goals

1. Reduce scheme discovery time from hours to under 2 minutes
2. Make eligibility reasoning transparent and verifiable — every match cites its source
3. Guide users from eligibility verdict to submitted application in a single session
4. Work for users with limited digital literacy — plain language, voice support, printable output
5. Be trustworthy — never hallucinate eligibility; flag uncertainty explicitly

3.2 Hackathon-specific success metrics

Metric	Target for demo	How to measure
Scheme match accuracy	>90% for profiles in test set	Manual review against scheme PDFs
Eligibility explanation quality	Every verdict has a source citation	Inspect output for citation presence
End-to-end flow completion	User reaches checklist from text input in <3 min	Timed demo run
Scheme coverage	50+ central + 20+ Gujarat state schemes	Count of ingested scheme PDFs
Conflict detection	Correctly flags 3+ known scheme conflicts	Pre-defined test cases

4. User Personas

Persona	Profile	Primary need	Key constraint
Ramesh, farmer	55yr, 2 acres wheat, Mehsana district, Gujarat, income ~1.2L/yr	Find agriculture and crop insurance schemes	Low digital literacy, uses basic smartphone
Priya, widow	34yr, 2 children, no income, BPL card, urban slum	Housing, food security, single-parent support schemes	No printer, depends on CSC for applications
Arjun, student	19yr, SC category, family income 90K/yr, engineering aspirant	Scholarship and fee-waiver schemes	Has smartphone but confused by multiple portals
Meena, CSC operator	28yr, runs Common Service Centre, serves 200+ citizens/month	Quick lookup tool to help citizens apply	Needs offline mode, printable summaries
Rahul, NGO worker	32yr, works with migrant labour in Surat	Batch eligibility check for 50 workers at once	Needs shareable output, WhatsApp-compatible

5. Core Feature Specifications

The following features represent the non-negotiable foundation. Without any one of these, PolicyPilot does not function as described. They are grouped by the user journey phase they serve.

1 Input & profile collection

How the citizen describes their situation

Structured onboarding form

Core | UX layer

Multi-step form that collects all fields government schemes actually filter on: state, district, age, annual income, family size, occupation type, land ownership, BPL card status, caste category, disability status, gender.

Implementation detail

One category per step. Save profile to localStorage so user doesn't re-enter. Show 'Your profile unlocks 12 more schemes if you add caste category' to encourage completion. Progress bar throughout.

Free-text situation description

Core | AI-powered

User types or speaks their situation in plain language. LLM extracts structured profile fields from this text. Show extracted fields for user confirmation before search.

Implementation detail

Prompt: extract JSON with fields {occupation, income_approx, family_size, gender, state, intent, special_categories}. Show side-by-side: 'We understood: farmer, income ~1.2L, Gujarat, 2 dependents. Is this right?' before proceeding.

Document upload + OCR extraction

Core | AI-powered

User uploads Aadhaar, ration card, or income certificate. System reads the document and pre-fills profile fields. User confirms each extracted value.

Implementation detail

Claude Vision or pytesseract for OCR. Extract: name, DOB, address, income, family members, caste. Show confidence per field. Reduces form fill time from 10 min to under 2 min.

2 Scheme knowledge base (RAG core)

PDF ingestion, chunking, embedding, retrieval

PDF ingestion pipeline

Core | RAG layer

Parse, chunk, and embed actual government scheme PDFs into a vector database. Store with rich metadata for filtering.

Implementation detail

PyMuPDF or pdfplumber for extraction. Chunk by logical section (eligibility, benefit, application process, documents needed) — NOT fixed token windows. Embed with Gemini embeddings. Metadata: scheme_id, ministry, state, section_type, page_number, ingestion_date.

Eligibility rule parser

Core | RAG layer | AI-powered

Extract structured eligibility conditions from scheme text: income thresholds, age ranges, state restrictions, caste categories,

Implementation detail

LLM extracts conditions into JSON: [{field: 'annual_income', operator: '<', value: 200000}, {field: 'state', operator: 'in', value: ['Gujarat','Maharashtra']}]. This enables hard-filter before semantic search — 10x more accurate than RAG alone.

occupational requirements. Store as queryable rules.	
Hybrid search (keyword + semantic) *Core | RAG layer* Combine BM25 keyword matching with vector similarity search for retrieval. Re-rank with cross-encoder. Pure vector search fails on government text with exact numbers and proper nouns.	**Implementation detail** BM25 via rank_bm25 for exact terms (income figures, scheme names, ministry names). Vector search via ChromaDB/FAISS for semantic matches. Cross-encoder re-ranking on top-20 candidates. Hybrid score = 0.6 * bm25 + 0.4 * vector.
Scheme metadata catalog *Core | RAG layer* Structured database of all schemes: ministry, state/central, benefit type, benefit amount, application portal URL, application deadline, last updated date.	**Implementation detail** Seed with 50 central + 20 Gujarat state schemes. Fields: scheme_id, name, ministry, level, benefit_type, benefit_value_inr, portal_url, application_deadline, pdf_version_date. Source from MyScheme.gov.in + manual curation.

3 Eligibility matching & explanation

The core intelligence — who qualifies and why

Multi-stage eligibility engine *Core | AI-powered | RAG layer* Three-stage pipeline: hard rule filter (eliminates impossible schemes), semantic relevance ranking, LLM deep-verification on top candidates.	**Implementation detail** Stage 1: structured rule evaluation — eliminates schemes where user provably doesn't qualify (wrong state, income above threshold). Stage 2: semantic search on user situation. Stage 3: LLM carefully checks all sub-conditions for top 10 results. Returns: qualified[], maybe_qualified[], ineligible[].
Cited eligibility explanation *Core | RAG layer | Trust feature* Every eligibility verdict shows the exact source clause — scheme name, section, page number, quoted text. Green = met, Red = blocking, Yellow = uncertain.	**Implementation detail** Every RAG response includes chunk metadata. Display: 'You qualify because: [PM-KISAN gazette, Section 3.1, Page 4]: Annual income of farming families not exceeding Rs. 2,00,000.' Users can tap to read full source paragraph.
Confidence scoring *Core | UX layer* Three-tier confidence per scheme match: High (all conditions verifiably met), Medium (one field missing or approximate), Low (LLM inferred, no direct clause found).	**Implementation detail** High: all hard conditions matched from user profile data. Medium: most conditions met, one profile field was approximate (e.g. income self-reported). Low: eligibility inferred from scheme context without explicit clause. Show 'Add income certificate to upgrade to High confidence.'

4 Application guidance

Getting from eligible to applied

Personalized application checklist

Core | AI-powered | UX layer

Step-by-step to-do list for each scheme. Steps extracted from PDF application section, then personalized: skip steps user has already completed based on their uploaded documents.

Implementation detail

LLM extracts application steps from scheme PDF. Personalize: if user has Aadhaar-linked bank account in profile, skip that step. Each step: action label, location (portal URL or office type), estimated time, cost. Mark complete as user progresses — persist across sessions in SQLite.

Document requirement checker

Core | AI-powered

List every document required for each scheme. Cross-check against user's uploaded documents. For each missing document: what it is, which authority issues it, how to apply, typical processing time, fee.

Implementation detail

LLM extracts required documents from scheme PDF. Compare against user's doc uploads. For missing docs: 'Income certificate — issued by Tehsildar/Mamlatdar office, application free, 7-10 working days.' Add nearest office lookup using district from user's Aadhaar.

Official portal deep links

Core | UX layer

Direct link to the exact application page on the official government portal — not just the home page. Show portal login requirements upfront.

Implementation detail

Manually verified URLs for all 70+ schemes in catalog. Detect if portal requires Aadhaar/DigiLocker authentication and warn user. Show application window status: 'Open — closes March 31' vs 'Closed — opens next Kharif season.'

Printable summary card

Core | UX layer

One-page PDF summary: scheme name, benefit value, user's eligibility reason with citation, documents needed, application steps, portal URL. Designed for CSC operators to print and hand to citizen.

Implementation detail

Generate with ReportLab or WeasyPrint. Print-optimized CSS. Works offline once cached. WhatsApp share button sends plain-text version for feature phone users. Show QR code linking to the application portal.

5 Discovery & comparison

Helping users understand the landscape

Scheme search & browse

Core | UX layer

Search bar and category filters for users who want to explore beyond profile-matched results. Filter by: category, state/central, benefit type, application status.

Implementation detail

Filters: Agriculture, Housing, Education, Health, Employment, Women & Child, Social Security. Sort by: benefit value, deadline, ease of application. 'New this month' and 'Closing soon' sections.

Benefit comparison table

UX layer | AI-powered

Side-by-side comparison of matched schemes: benefit amount (normalized to annual INR), application effort score, deadline, eligibility confidence.

Implementation detail

Normalize all benefits to annual INR equivalent. Application effort: Easy (<5 docs, fully online), Medium (5-8 docs, partial office visit), Hard (10+ docs, multiple office visits). Show 'Best ROI' badge on highest benefit / lowest effort scheme.

RAG-grounded chatbot

AI-powered | UX layer

After matching, user can ask follow-up questions. Answers grounded in scheme PDFs — not LLM general knowledge. Maintains user's profile context throughout.

Implementation detail

System prompt includes: user profile JSON + retrieved scheme chunks. Every answer cites source. Handles: 'What if my income goes up?', 'Can I apply for both?', 'What does beneficiary mean in this context?' Fallback: 'I cannot find a definitive answer — please call the ministry helpline at [number].'

6. Differentiation & Moat Features

The following features go beyond MVP functionality to create defensible competitive advantages — either through data that compounds with usage, technical complexity that takes significant effort to replicate, or trust built through transparency that becomes a brand.

6.1 Data moat features

These features become more valuable the more users interact with PolicyPilot. They cannot be replicated by a new entrant without real adoption.

[DATA MOAT] Scheme success rate tracker

Crowdsource real approval rates. Show: 'Of 120 users with your profile who applied for PMAY-G, 74 got approved (62%).'

Why this is a moat: *Nobody else has this data. Every user who completes their application journey adds a data point. After 1,000 users, this is irreplaceable. Competitors starting fresh have zero data.*

How to build it: After application submission, ask: 'Did you get approved?' Store: scheme_id + anonymized profile bucket + outcome. Aggregate and display as approval rate for 'profiles like yours.' Bucket by: state + income_band + caste + occupation_type.

Hackathon angle: For hackathon: seed with 25-30 realistic data points across 5-6 common schemes. Judges will immediately grasp the long-term value. Demo line: 'Users like you have a 68% success rate for PM-KISAN — here is what the 32% who got rejected did wrong.'

[DATA MOAT] Rejection reason database

Collect real rejection reasons, categorize them, and surface the most common traps before a user applies.

Why this is a moat: *Government rejection letters are cryptic. Real rejection reasons from real users become a searchable knowledge base that no PDF ingestion can replace.*

How to build it: Optional post-rejection flow: 'Were you rejected? Tell us why.' Free text → LLM categorizes into: income_mismatch / wrong_documents / deadline_missed / already_beneficiary / name_mismatch / other. Tag by scheme_id. Display as 'Common rejection reasons' card in the application guide.

Hackathon angle: Pre-seed with 5 known rejection patterns per major scheme (researchable from government help forums). Demo: 'Warning: 43% of PMAY-G rejections occur because the applicant name on Aadhaar does not match the bank account. Check yours now.'

6.2 Complexity moat features

These require real engineering effort and scheme-domain knowledge. They cannot be built in a weekend by a team that hasn't invested in understanding government policy logic.

[COMPLEXITY MOAT] Multi-scheme conflict & overlap detector

Automatically detect when two matched schemes have conflicting rules, duplicate benefits, or where applying for one disqualifies you from another.

Why this is a moat: *This requires parsing and reasoning across multiple PDFs simultaneously. Pure RAG cannot do this — it needs structured rule comparison logic layered on top of retrieval.*

How to build it: After matching N schemes, run LLM pairwise analysis: 'Does qualifying for Scheme A affect eligibility for Scheme B?' Flag three conflict types: (1) mutual exclusivity — 'beneficiary of any other

housing scheme is ineligible', (2) benefit duplication — same benefit type, check if both allowed simultaneously, (3) income interaction — some benefits count as income for other schemes' thresholds.
Hackathon angle: PMAY and Gujarat state housing scheme have mutual exclusivity. PM-KISAN and certain state farmer schemes have income interaction. Hard-code 4 known conflicts as demo test cases. Show as: 'Conflict detected: Applying for PMAY-G will disqualify you from Gujarat Mukhyamantri Grih Yojana — which offers 40% higher subsidy. Recommended: apply for state scheme first.'

[COMPLEXITY MOAT] Benefit calculation engine

Don't just say 'you qualify for PMFBY crop insurance.' Calculate the actual rupee amount based on land size, crop type, and district-level premium rate.

Why this is a moat: *Each scheme has a different calculation formula. Encoding each one correctly requires actually reading and understanding the scheme document. This is manual work per scheme — not automatable.*

How to build it: For each major scheme, build a parameterized calculator. PMFBY: crop_type + land_area_acres + district → premium_amount + max_claim. PM-KISAN: flat Rs. 6,000/yr, no calculation needed. PMAY: loan_amount × subsidy_rate based on income slab (EWS/LIG/MIG). NMMS scholarship: board exam marks + income → award amount. LLM extracts missing parameters in conversation.

Hackathon angle: Show 3 calculators working live in demo. PMFBY is most impressive — output is highly personalized and the numbers are often surprising to users. Demo: 'Based on your 2 acres of wheat in Mehsana district, your PMFBY premium is Rs. 847 and your maximum insured amount is Rs. 42,000.'

[COMPLEXITY MOAT] Scheme change diff tracker

When a scheme PDF is updated, automatically detect what changed — eligibility thresholds, benefit amounts, deadlines — and notify users who are newly eligible.

Why this is a moat: *Government schemes change constantly. No platform tracks this systematically. Users currently find out only after rejection. This makes PolicyPilot a source of ongoing value, not a one-time tool.*

How to build it: Store previous version of each scheme's extracted rule JSON. On re-ingestion of updated PDF, diff the structured rules. Detect: threshold changes, new states added, benefit amount updates, deadline extensions. Alert users previously marked as ineligible if new rules make them eligible. Store version history per scheme.

Hackathon angle: Show a 'What changed' log per scheme — even a simple version history display demonstrates operational thinking. Demo: 'This scheme was updated 3 days ago: income limit raised from Rs. 1.5L to Rs. 2L. 240 previously ineligible users now qualify — you are one of them.'

6.3 Trust moat features

Trust is built incrementally through transparency, citations, and consistent accuracy. Once users trust a platform for high-stakes decisions like welfare applications, switching costs are high.

[TRUST MOAT] Source-grounded answer confidence UI

Every AI answer shows which PDF it came from, which section, confidence level, and a staleness flag if the scheme PDF is over 6 months old.

Why this is a moat: *Other tools hallucinate and users cannot tell. PolicyPilot's transparency becomes its brand identity — 'the one that shows its work.' Once users learn to look for citations, they will distrust tools that don't show them.*

How to build it: Every RAG response includes retrieved chunk metadata. Display inline: [Source: PM-KISAN Gazette, Section 4.2, Page 7, Dec 2023]. Add freshness indicator: 'PDF last updated 14 months ago — verify current rules on official portal.' Color-code: green (multiple sources agree), yellow (single source), orange (LLM inference only).

Hackathon angle: Most teams show answers. PolicyPilot shows answers with proof. The visual difference in the demo is immediately legible without explanation. Side-by-side comparison: competitor says 'You qualify.' PolicyPilot says 'You qualify — [PM-KISAN gazette, Section 3.1, Page 4, verified Jan 2025].'

[TRUST MOAT] **Offline-first CSC mode**

A simplified mode for Common Service Centre operators: works without internet, caches top 100 schemes, generates printable summaries.

Why this is a moat: *India has 5+ lakh CSCs serving 50 crore rural citizens. If PolicyPilot becomes the tool CSC operators trust, distribution is solved — government workers become the distribution channel.*

How to build it: PWA with service workers. Pre-cache top 100 schemes for selected state on first load. Sync eligibility verdicts when online. Print-optimized CSS for application summaries. No login required in CSC mode — operator selects state, enters citizen profile, gets matched schemes and printable checklist.

Hackathon angle: Show phone demo: open app, toggle airplane mode, search a scheme — still works. Show print preview of summary card. Judges from rural India backgrounds will feel this immediately. Demo narrative: 'A CSC operator in a village with no internet can still serve 20 citizens a day with PolicyPilot.'

7. Technical Architecture

7.1 Stack overview

Layer	Technology	Rationale
Backend API	Python + FastAPI	Fast to build, async support, easy to deploy
Frontend	React + TailwindCSS	Component reuse, mobile-responsive, fast iteration
Vector store	ChromaDB (dev) / FAISS (prod)	Local-first, no infra cost, portable
LLM	Gemini 1.5 Pro via API	Long context window for full PDF ingestion, free tier available
Embeddings	Gemini text-embedding-004	Same API key, strong multilingual performance
OCR	Pytesseract + PIL	Free, local, handles Aadhaar/ration card image formats
Keyword search	rank_bm25 (Python)	Lightweight, no additional infra needed
Database	SQLite	Zero config, perfect for hackathon and single-instance deploy
PDF generation	ReportLab or WeasyPrint	Printable application summaries and pre-filled forms
Deployment	Railway / Render (free tier)	One-click deploy, sufficient for hackathon demo traffic

7.2 RAG pipeline architecture

The RAG pipeline is the technical core. The following stages run in sequence during both ingestion and query time:

Ingestion pipeline (offline, run once per scheme PDF)

6. PDF parsing: extract text using PyMuPDF, preserve section headers and page numbers
7. Semantic chunking: split at section boundaries (Eligibility, Benefits, Application Process, Required Documents) rather than fixed token counts
8. Rule extraction: LLM pass to extract structured eligibility conditions into JSON per chunk
9. Embedding generation: embed each chunk using Gemini text-embedding-004
10. Storage: save chunk text, embedding vector, metadata (scheme_id, section, page, rules JSON) to ChromaDB
11. Metadata catalog update: upsert scheme-level record with name, ministry, benefit, portal URL, deadline

Query pipeline (real-time, per user request)

12. Profile normalization: convert user profile + free text into structured query fields
13. Hard filter: apply structured rules to eliminate schemes where user is definitively ineligible

14. Hybrid retrieval: BM25 + vector search on remaining schemes, merge and re-rank top 20
15. LLM verification: for top 10 candidates, prompt LLM to carefully evaluate all eligibility sub-conditions against user profile
16. Response generation: for each verified match, generate explanation with cited source chunks
17. Confidence scoring: assign High/Medium/Low based on data completeness and source quality

7.3 Data model

Entity	Key fields
Scheme	scheme_id, name, ministry, state_or_central, category, benefit_type, benefit_value_inr, portal_url, deadline, pdf_path, last_updated
SchemeChunk	chunk_id, scheme_id, section_type, page_number, text, embedding_vector, rules_json
UserProfile	profile_id, state, district, age, annual_income, family_size, occupation, land_acres, bpl_card, caste_category, disability, gender, created_at
MatchResult	result_id, profile_id, scheme_id, confidence, eligibility_verdict, cited_chunks, checklist_json, created_at
ApplicationProgress	progress_id, profile_id, scheme_id, steps_completed, docs_uploaded, status, updated_at
OutcomeRecord	outcome_id, profile_id, scheme_id, applied_at, result (approved/rejected/pending), rejection_reason_category

8. Scope — MVP vs Post-Hackathon

Feature	MVP (Hackathon)	Post-hackathon v2
Scheme coverage	50 central + 20 Gujarat schemes	All states, 500+ schemes, auto-ingestion
Language support	English + Hindi (basic)	22 scheduled languages via translation layer
Voice input	Optional stretch goal	Full vernacular voice with regional ASR
User accounts	LocalStorage / session only	Full auth, multi-device sync
Form pre-fill	Downloadable pre-filled PDF	Direct API submission to government portals
Scheme diff tracking	Demo concept only	Automated PDF re-ingestion with change detection
Success rate data	Seeded dummy data for demo	Real crowdsourced outcome data
CSC offline mode	PWA with service workers	Dedicated CSC operator dashboard
Notification system	None	SMS/WhatsApp alerts for deadline reminders, eligibility changes
Analytics	None	Usage patterns to improve scheme ranking and recommendation quality

9. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
LLM hallucinates eligibility	Medium	Critical — user applies for scheme they don't qualify for	3-stage pipeline with hard rule filter first; always show source citation; Low confidence label for inferred matches
Scheme PDFs outdated	High — schemes change frequently	High — wrong information given to user	Display PDF ingestion date prominently; freshness warning if >3 months old; always link to official portal
OCR fails on low-quality uploads	Medium	Medium — profile incomplete	Fallback to manual form entry; show confidence per extracted field; allow user to correct any field
Portal URLs change or go down	Medium	Medium — user can't find application page	Weekly automated URL health check; fallback to ministry homepage; last-verified date shown

Risk	Likelihood	Impact	Mitigation
User provides incorrect income	High	Medium — wrong eligibility verdict	Caveat all income-based verdicts; ask user to verify with income certificate; Medium confidence until document uploaded

10. Hackathon Demo Script

The demo should run in exactly 3 minutes. Each beat is designed to produce an audible reaction from the judges.

> **Demo setup**
> Pre-load: a farmer profile (Ramesh, 55yr, Mehsana, Gujarat, 2 acres wheat, income 1.2L, SC category). Have 5 scheme PDFs pre-ingested including PMFBY, PM-KISAN, PMAY-G, and two that have a known conflict.

Minute	Action	Expected reaction
0:00–0:30	Type situation in plain text: 'Main ek kisan hoon, Mehsana Gujarat mein, 2 acre gehun hai, income around 1.2 lakh, SC category.' Show LLM extraction: 'We understood: farmer, Mehsana district, 2 acres wheat, SC, income ~1.2L. Correct?' Click confirm.	Judges see natural language → structured data. First wow moment.
0:30–1:15	Show matched schemes — 5 results with confidence levels. Click PM-KISAN match. Show cited explanation: 'You qualify because [PM-KISAN gazette, Section 3.1]: annual income of farming families not exceeding Rs. 2,00,000.' Toggle to PMFBY and show benefit calculation: 'Your premium: Rs. 847. Max claim: Rs. 42,000.'	Judges see citations + personalized numbers. Trust established.
1:15–1:45	Show conflict detection card: 'Warning — PMAY-G and Gujarat Mukhyamantri Grih Yojana conflict. Applying for PMAY-G will disqualify you from state scheme which offers 40% higher benefit. Recommendation: apply for state scheme first.'	Strongest demo moment. No other team will show this.
1:45–2:30	Open application checklist for PM-KISAN. Show 6 steps, 3 already marked complete based on uploaded Aadhaar. Show missing document: 'Land record (Jamabandi) — get from district collectorate, free, 5 days.' Show direct link to PM-KISAN portal application page.	Judges see end-to-end guidance, not just discovery.
2:30–3:00	Toggle phone to airplane mode. Open CSC mode. Search 'housing scheme' — results still appear from cache. Show print preview of application summary card. Re-enable WiFi. Close with: 'This is PolicyPilot — from I don't know what I qualify for, to ready to apply, in under 3 minutes.'	Offline mode closes the demo with the distribution story.

11. Team & Build Plan

11.1 Recommended role split (2-person team)

Person	Owns	Parallel workstream
Builder A (backend/ML)	RAG pipeline, PDF ingestion, eligibility engine, FastAPI	Scheme data curation — source and ingest 70 PDFs
Builder B (frontend/UX)	React UI, profile form, results display, checklist view	Demo data seeding — success rates, rejection reasons, known conflicts

11.2 24-hour build schedule

Hours	Milestone
H0–H4	RAG pipeline working end-to-end with 10 test schemes. Profile form functional. Basic match results displaying.
H4–H8	Cited explanations working. Confidence scoring live. 50 schemes ingested. Document checker functional.
H8–H14	Conflict detector built and tested with 3 known conflicts. Benefit calculator for PMFBY + PM-KISAN. Application checklist with portal links.
H14–H18	CSC offline mode (PWA). Success rate UI with seeded data. Printable summary card. UI polish.
H18–H22	Demo data seeding. End-to-end test run. Edge case fixing. Rejection reasons seeded for 5 major schemes.
H22–H24	Demo rehearsal x3. Slide deck. Backup demo plan if live system fails.

12. Appendix — Scheme Coverage List

The following schemes form the initial knowledge base. All are available as public PDFs from India.gov.in, MyScheme.gov.in, and respective ministry websites.

Central schemes (priority ingestion)

Scheme	Ministry	Category	Key eligibility
PM-KISAN	Agriculture	Agriculture	Small/marginal farmers, income <2L
PMFBY	Agriculture	Agriculture	Farmers with crop loans or voluntary enrollment
PMAY-G	Rural Development	Housing	BPL/EWS rural households, no pucca house
PMAY-U	Housing & Urban	Housing	EWS/LIG urban households, first-time owners
PM-SVANidhi	Housing & Urban	Employment	Street vendors with certificate

Scheme	Ministry	Category	Key eligibility
PMJDY	Finance	Financial inclusion	Unbanked individuals
Mudra Yojana	Finance	Employment	Non-farm micro-enterprises, loan up to 10L
NREGA	Rural Development	Employment	Rural households, unskilled manual work
NSAP (IGNOAPS etc.)	Rural Development	Social security	BPL elderly/widows/disabled
PM Ujjwala	Petroleum	Energy	BPL women, no LPG connection
Ayushman Bharat	Health	Health	Bottom 40% by SECC, up to 5L cover/year
NMMS Scholarship	Education	Education	Class 9-12, income <1.5L, merit-based
NSP (various)	Education	Education	SC/ST/OBC/Minority students, various income limits
Startup India Seed Fund	DPIIT	Entrepreneurship	DPIIT-recognized startups, early stage

Gujarat state schemes (demo-specific)

Scheme	Category	Key eligibility
Mukhyamantri Grih Yojana	Housing	Gujarat resident, income <6L, no own house
Kisan Suvidha	Agriculture	Gujarat farmer, Aadhar-linked land record
Manav Garima Yojana	Social welfare	SC category, self-employment tools
Vidya Saathi	Education	Gujarat student, class 9+, income <2L
Vahali Dikri Yojana	Women & child	Girl child, Gujarat resident, born after 2019

PolicyPilot PRD v1.0 — End of Document